

Relatório de Pesquisa Avançada: Viabilidade Técnica e Implementação de Emulação Android em Instâncias AWS EC2 Virtualizadas (Non-Metal)

Sumário Executivo

Este relatório técnico apresenta uma análise exaustiva sobre a viabilidade, arquitetura e implementação de ambientes de emulação Android na infraestrutura de nuvem da Amazon Web Services (AWS), estritamente focado em instâncias virtualizadas de baixo custo, como as famílias t3.large e t4g.large. A pesquisa responde à necessidade crítica de reduzir os custos operacionais associados à emulação móvel, que tradicionalmente exige instâncias *bare metal* de alto valor (ex: c5.metal) devido à necessidade de virtualização aninhada (*nested virtualization*).

A análise identifica que, embora as instâncias virtualizadas padrão da AWS (baseadas no Nitro System) não suportem a aceleração de hardware KVM (Kernel-based Virtual Machine) necessária para o emulador padrão do Android Studio (AVD), é possível contornar essa limitação técnica através da **containerização do sistema operacional Android**. Diferente da virtualização clássica, que exige emulação de hardware, a containerização permite que o Android compartilhe o kernel Linux da instância host, eliminando a barreira do hipervisor.

O documento detalha a superioridade técnica e econômica da arquitetura ARM64 (família **AWS Graviton / T4g**) sobre a arquitetura x86 (família **Intel / T3**) para esta finalidade específica. A execução nativa de instruções ARM no processador Graviton elimina a necessidade de camadas de tradução binária (como *libhoudini*), resultando em uma performance significativamente superior e uma redução de custo de aproximadamente 20% em comparação às instâncias T3 equivalentes.

Além disso, o relatório fornece um guia de implementação profundo, abordando a complexidade da integração de módulos de kernel específicos do Android (binder, ashmem, memfd) em distribuições Linux de servidor (Ubuntu Server), o gerenciamento de créditos de CPU em instâncias *burstable*, e as estratégias de renderização gráfica via software para instâncias *headless* (sem GPU dedicada).

1. Contextualização Tecnológica e O Dilema da

Virtualização em Nuvem

A execução de sistemas operacionais móveis em ambientes de servidor apresenta desafios únicos que diferem substancialmente da virtualização de servidores Linux ou Windows tradicionais. Para compreender as restrições impostas pelas instâncias da família T3 e similares, é imperativo dissecar a arquitetura de virtualização subjacente da AWS e os requisitos do Android.

1.1. A Arquitetura de Virtualização da AWS e o Sistema Nitro

A infraestrutura moderna da AWS é construída sobre o **AWS Nitro System**, uma plataforma que delega as funções de virtualização de rede, armazenamento e segurança para hardware dedicado, permitindo que a CPU principal do servidor seja dedicada quase inteiramente à instância do cliente.

1.1.1. Limitações de Virtualização Aninhada (Nested Virtualization)

A maioria dos emuladores Android, especificamente o AVD (Android Virtual Device) fornecido pelo Google, baseia-se no QEMU-KVM. O KVM é um módulo do kernel Linux que transforma o sistema operacional em um hipervisor do tipo 1, permitindo que a máquina virtual (o dispositivo Android) execute instruções diretamente na CPU física através das extensões de virtualização de hardware: **Intel VT-x** (para processadores Intel) ou **AMD-V** (para processadores AMD).¹

Em um ambiente de nuvem padrão (instâncias não-metal como t3.large, m5.large, c5.large), o sistema operacional do usuário (Guest OS) já está rodando dentro de uma máquina virtual gerenciada pelo hipervisor da AWS (KVM modificado ou Nitro Hypervisor). Para que o usuário possa rodar o KVM dentro de sua instância, o hipervisor da AWS precisaria expor as extensões VT-x/AMD-V para o Guest OS. Esse recurso é denominado **Virtualização Aninhada** (*Nested Virtualization*).

A pesquisa confirma inequivocamente que a AWS **não suporta virtualização aninhada** nas instâncias virtualizadas das famílias T3, M5, C5 e suas variantes.¹ O suporte a essas instruções é restrito exclusivamente às instâncias *bare metal* (.metal), que garantem acesso direto ao hardware físico sem a interposição de um hipervisor.

1.1.2. Consequências para o Emulador Tradicional

Ao tentar executar um emulador baseado em KVM (como o emulador padrão do Android Studio) em uma instância t3.large:

1. **Falha de Inicialização:** O emulador verificará a presença do dispositivo `/dev/kvm`. Se não encontrado, ou se as permissões de acesso ao hardware virtual não forem concedidas, ele falhará.
2. **Fallback para Emulação de Software (TCG):** O QEMU pode reverter para o modo TCG

(*Tiny Code Generator*), onde cada instrução da CPU do sistema convidado é traduzida via software para a CPU do host. Isso resulta em uma degradação de desempenho catastrófica, tornando o sistema inutilizável (tempos de boot superiores a 20 minutos e latência de interação de vários segundos).⁵

Portanto, a premissa de utilizar o emulador padrão do Android Studio em instâncias T3 é tecnicamente inviável para qualquer cenário produtivo.

1.2. A Mudança de Paradigma: Android em Container (Android-in-Container)

Para superar a limitação da virtualização aninhada sem incorrer nos custos proibitivos das instâncias *bare metal*, a solução técnica reside na mudança do modelo de isolamento: da virtualização de hardware para a **containerização**.

O Android é construído sobre o kernel Linux. Isso permite, teoricamente, que o *userspace* do Android (a camada de aplicação, bibliotecas e runtime) seja executado diretamente sobre o kernel de uma distribuição Linux padrão (como Ubuntu), isolado através de *namespaces* e *cgroups*, da mesma forma que o Docker isola aplicações web.

1.2.1. Vantagens da Abordagem via Container

- **Eliminação do Hipervisor:** Não é necessário virtualizar hardware. O container Android faz chamadas de sistema (*syscalls*) diretamente ao kernel do host. Isso contorna completamente a falta de suporte a VT-x/AMD-V nas instâncias T3/T4g.⁶
- **Densidade e Performance:** A sobrecarga (*overhead*) de um container é mínima comparada a uma VM completa. Isso é crucial para instâncias com recursos limitados, como a t3.large (2 vCPUs, 8GB RAM).
- **Elasticidade:** Containers iniciam em segundos, ao contrário de minutos para VMs completas, favorecendo pipelines de CI/CD dinâmicos.

1.2.2. Projetos Líderes: Redroid vs. Anbox vs. Waydroid

A pesquisa identificou três principais tecnologias neste espaço:

1. **Redroid (Remote Android):** A solução mais robusta e ativa para ambientes de servidor *headless* (sem monitor). Otimizado para Docker, suporta aceleração gráfica via GPU ou software e é agnóstico em relação à arquitetura (x86 e ARM).⁶
2. **Anbox Cloud:** Solução comercial da Canonical. Embora poderosa, a versão open-source original (Anbox) foi descontinuada e o foco atual é o produto empresarial gerenciado, que pode ter custos de licenciamento adicionais no AWS Marketplace.¹⁰
3. **Waydroid:** Sucessor espiritual do Anbox, utiliza a tecnologia Wayland para integração gráfica. No entanto, sua dependência do Wayland torna a configuração em servidores *headless* na nuvem (que não rodam um ambiente gráfico desktop por padrão) complexa

e propensa a erros.¹⁰

Conclusão da Seleção: O **Redroid** emerge como a solução técnica ideal para a solicitação do usuário, oferecendo flexibilidade de Docker, suporte ativo e documentação para implantação em nuvem.

2. Análise Comparativa de Arquitetura de Hardware: x86 vs. ARM

A escolha entre as famílias de instâncias t3 (x86_64) e t4g (ARM64) é a decisão arquitetural mais impactante deste projeto. Embora o usuário tenha solicitado explicitamente "família t3.large", a análise técnica demonstra que a família t4g é superior em todas as métricas relevantes para emulação Android.

2.1. O Ecossistema Android e a Dominância ARM

O Android é um sistema operacional projetado primariamente para arquitetura ARM. A vasta maioria dos dispositivos físicos (smartphones e tablets) utiliza processadores ARM. Consequentemente, a maioria dos aplicativos (APKs) disponíveis na Google Play Store contém código nativo compilado para ARM (armeabi-v7a ou arm64-v8a).

2.2. Família T3 (Intel x86_64) e o Custo da Tradução

As instâncias **T3** utilizam processadores Intel Xeon Platinum da série 8000 (arquitetura Skylake ou Cascade Lake). Ao rodar o Android nesta arquitetura:

1. **Sistema Operacional:** O Android-x86 (versão do OS portada para Intel) roda nativamente e com boa performance.
2. **Aplicativos:** Se o aplicativo a ser testado ou executado possuir apenas bibliotecas nativas ARM (o que é comum em jogos e apps comerciais complexos), ele não rodará diretamente na CPU x86.
3. **A Necessidade de Tradução Binária:** Para executar binários ARM em x86, é necessário utilizar uma camada de tradução de instruções em tempo real, conhecida como **Native Bridge**. As soluções mais comuns são o **libhoudini** (proprietário da Intel) e o **libndk_translation** (proprietário do Google).¹³

Desvantagens da Tradução (libhoudini/libndk):

- **Penalidade de Performance:** A tradução de instruções ARM para x86 em tempo real consome ciclos de CPU significativos (overhead), reduzindo a capacidade de processamento disponível para a aplicação em si.
- **Instabilidade:** A integração do libhoudini em containers Docker (como Redroid) é complexa, frequentemente envolvendo a extração de binários proprietários de imagens

do emulador oficial, o que pode violar termos de serviço ou gerar instabilidade técnica.¹⁵

- **Consumo de Memória:** O processo de tradução exige mais memória RAM, pressionando os 8GB disponíveis na instância t3.large.

2.3. Família T4g (AWS Graviton2 - ARM64) e a Execução Nativa

As instâncias **T4g** são alimentadas pelos processadores AWS Graviton2, baseados na arquitetura ARM Neoverse N1 de 64 bits.

1. **Execução Nativa:** Tanto o sistema operacional Android quanto os aplicativos ARM rodam nativamente na CPU, sem qualquer camada de tradução intermediária.¹⁶
2. **Eficiência Energética e Custo:** A arquitetura RISC dos processadores ARM é inerentemente mais eficiente. A AWS repassa essa eficiência no preço: as instâncias T4g são tipicamente 20% mais baratas que as T3 equivalentes.
3. **Simplificação do Stack de Software:** Não é necessário configurar libhoudini ou libndk. O container Redroid para ARM64 (redroid:android12-arm64) funciona "out-of-the-box" com máxima compatibilidade de aplicativos.

2.4. Comparativo de Especificações e Custos (Região us-east-1)

A tabela abaixo sintetiza a comparação entre as opções analisadas, evidenciando a vantagem da T4g.

Métrica	T3.large (x86_64)	T3a.large (AMD x86)	T4g.large (ARM64)	C5.metal (Referência)
vCPUs	2	2	2	96
Memória	8 GiB	8 GiB	8 GiB	192 GiB
Processador	Intel Xeon Platinum 8000	AMD EPYC 7000 Series	AWS Graviton2 (ARM)	Intel Xeon Scalable
Preço On-Demand (Hora)	~\$0.0832	~\$0.0752	~\$0.0672	~\$4.08
Custo Mensal (730h)	~\$60.74	~\$54.90	~\$49.06	~\$2,978.40
Arquitetura Android	x86 (Requer libhoudini para	x86 (Requer libhoudini para	ARM64 Nativo	x86 (Com KVM)

	apps ARM)	apps ARM)		
Suporte KVM	Não	Não	Não	Sim
Recomendação	Secundária	Secundária	Primária	Inviável (Custo)

Dados de preços baseados em valores públicos da AWS para a região US East (N. Virginia), sujeitos a alterações.¹⁹

Conclusão da Análise de Hardware: A instância **t4g.large** é a escolha recomendada. Ela atende ao requisito de "custo similar ou menor" que a T3, elimina a necessidade de tradução binária complexa e oferece performance superior para cargas de trabalho Android. O relatório focará na implementação nesta arquitetura, mantendo notas de compatibilidade para T3 caso o usuário tenha restrições rígidas de infraestrutura x86.

3. Engenharia do Kernel: O Coração da Containerização Android

Para que o Android funcione dentro de um container Docker em uma distribuição Linux comum (como Ubuntu), o kernel do host deve fornecer funcionalidades específicas que não são habilitadas por padrão em kernels de servidor. Esta é a etapa mais crítica e tecnicamente complexa da implementação.

3.1. O Mecanismo Binder (IPC)

O **Binder** é o mecanismo de Comunicação Inter-Processos (IPC) do Android. Diferente do IPC System V ou sockets Unix tradicionais, o Binder é otimizado para alta performance e baixa latência, permitindo que objetos passem entre processos.

- **Requisito:** O kernel deve carregar o módulo `binder_linux`.
- **Dispositivos:** O módulo deve expor os nós de dispositivo `/dev/binder`, `/dev/hwbinder` e `/dev/vndbinder` para que o *servicemanager* do Android possa operar.
- **Evolução (BinderFS):** Em kernels modernos (5.0+), o suporte ao Binder pode ser fornecido através do sistema de arquivos **binderfs**. Isso permite montar dinamicamente os dispositivos Binder, oferecendo maior flexibilidade e isolamento para múltiplos containers Android rodando no mesmo host.²³

3.2. Gerenciamento de Memória: Ashmem vs. Memfd

O Android utiliza memória compartilhada para passar grandes volumes de dados (como

buffers gráficos) entre processos sem cópia (zero-copy).

- **Ashmem (Anonymous Shared Memory):** Historicamente, o Android utilizava o driver ashmem. No entanto, este driver foi considerado obsoleto pela comunidade do kernel Linux e **removido definitivamente na versão 5.18**.⁸
- **Memfd (Memory File Descriptor):** O substituto moderno e padrão no Linux upstream. O Android 10 introduziu suporte parcial e o Android 12+ suporta totalmente o uso de memfd em vez de ashmem.
- **Impacto na AWS:** As imagens Ubuntu 22.04 e 24.04 na AWS frequentemente utilizam kernels atualizados (5.15, 6.2, 6.5+). Se o kernel for 5.18 ou superior, tentar carregar o módulo ashmem_linux resultará em erro fatal ("Module not found").
- **Solução:** Configurar o runtime do Android (Redroid) para utilizar memfd explicitamente através da propriedade androidboot.use_memfd=1. Isso elimina a necessidade de compilar módulos ashmem legados.²⁶

3.3. Desafios das AMIs AWS Padrão

As Amazon Machine Images (AMIs) oficiais, como o ubuntu-jammy-22.04-amd64-server ou suas variantes ARM64, vêm com o kernel otimizado linux-aws.

- **Problema:** Este kernel é enxuto e muitas vezes não inclui os módulos binder_linux no pacote principal, ou eles não estão habilitados na configuração de compilação.
- **Solução:** A instalação do pacote linux-modules-extra-aws ou linux-modules-extra-\$(uname -r) geralmente fornece os módulos necessários. Caso contrário, é necessário utilizar o DKMS (Dynamic Kernel Module Support) para compilar os módulos a partir do código fonte fornecido pelo projeto Redroid.²⁸

4. Guia de Implementação Técnica Passo-a-Passo

Este guia detalha a configuração de um ambiente de emulação Android utilizando **Redroid** em uma instância **AWS EC2 t4g.large** (ARM64), rodando **Ubuntu 22.04 LTS**. As instruções são adaptáveis para t3.large (x86) com as devidas ressalvas indicadas.

4.1. Provisionamento de Infraestrutura (AWS EC2)

1. **Console EC2:** Inicie o assistente de lançamento de instância ("Launch Instances").
2. **Nome:** Android-Emulator-T4g-Production.
3. **Imagem de SO (AMI):**
 - Selecione **Ubuntu**.
 - Versão: **Ubuntu Server 22.04 LTS (HVM), SSD Volume Type**.
 - **Arquitetura:** Selecione **64-bit (Arm)** para T4g. (Se usar T3, selecione 64-bit x86).
4. **Tipo de Instância:**
 - Selecione **t4g.large** (2 vCPUs, 8 GiB Memory).
 - **Nota:** A instância t4g.medium (4 GiB) é arriscada, pois o Android + Docker + OS Host

podem exceder a memória, acionando o OOM Killer (Out of Memory Killer) do Linux.

5. **Chave de Acesso (Key Pair):** Crie ou selecione uma chave .pem para acesso SSH.

6. **Configurações de Rede (Network):**

- Crie um novo Security Group.
- **Regra 1:** SSH (TCP 22) - Origem: My IP (Seu IP atual).
- **Regra 2:** Custom TCP (Porta 5555) - Origem: My IP.
 - **Aviso de Segurança Crítico:** A porta 5555 é usada pelo ADB (Android Debug Bridge). O ADB não possui autenticação robusta por padrão. Deixar essa porta aberta para 0.0.0.0/0 (qualquer lugar) expõe o emulador a ataques, instalação de malware e mineração de criptomoedas.⁹

7. **Armazenamento (Storage):**

- O padrão de 8GB é insuficiente para o Android.
- Aumente para **30 GiB** ou mais.
- Tipo de volume: **gp3** (General Purpose SSD) para melhor performance de I/O, crucial para o carregamento de apps e boot do sistema.³⁰

4.2. Preparação do Sistema Operacional e Kernel

Conecte-se à instância via SSH:

Bash

```
ssh -i "sua-chave.pem" ubuntu@<IP-PUBLICO-EC2>
```

Execute a preparação do ambiente. Este script automatiza a atualização, instalação do Docker e configuração dos módulos.

Bash

```
# 1. Atualização do Sistema
```

```
sudo apt-get update && sudo apt-get upgrade -y
```

```
# 2. Instalação de Dependências Essenciais e Docker
```

```
sudo apt-get install -y docker.io git kmod
```

```
sudo usermod -aG docker $USER
```

```
# (É recomendável fazer logout/login ou usar 'newgrp docker' para aplicar o grupo)
```



```
# 3. Preparação dos Módulos de Kernel (Binder)
# Tenta instalar os módulos extras oficiais da AWS para o kernel atual
sudo apt-get install -y linux-modules-extra-aws
sudo apt-get install -y linux-modules-extra-$(uname -r)
```

```
# 4. Ativação dos Módulos
# Binder é essencial para IPC.
sudo modprobe binder_linux devices="binder,hwbinder,vndbinder"
```

```
# Verificação
lsmod | grep binder
# Saída esperada: binder_linux <tamanho> <uso>
```

Plano de Contingência (Se modprobe falhar):

Se os pacotes acima não habilitarem o binder (comum em algumas AMLs customizadas), compile manualmente:

Bash

```
# Instalação de headers do kernel e ferramentas de compilação
sudo apt-get install -y dkms build-essential linux-headers-$(uname -r)
```

```
# Clonagem do repositório de módulos do Redroid
git clone https://github.com/remote-android/redroid-modules.git
cd redroid-modules
```

```
# Compilação e Instalação via DKMS
sudo make install
```

*Este passo garante que os módulos sejam recompilados automaticamente se o kernel for atualizado.*²⁸

4.3. Implantação do Container Redroid

A configuração do Docker determina a performance e a estabilidade do emulador.

Comando de Execução para t4g.large (ARM64 - Recomendado):

Bash

```
docker run -itd \
--rm \
--privileged \
--pull always \
--name redroid-instance \
-v ~/data-android:/data \
-p 5555:5555 \
redroid/redroid:12.0.0-latest \
androidboot.redroid_width=720 \
androidboot.redroid_height=1280 \
androidboot.redroid_dpi=320 \
androidboot.use_memfd=1 \
androidboot.redroid_gpu_mode=guest \
androidboot.redroid_fps=30
```

Análise dos Parâmetros:

- `--privileged`: Concede ao container acesso direto aos dispositivos do host (`/dev/binder`, etc). Indispensável.
- `-v ~/data-android:/data`: Persistência de dados. Se o container for recriado, os apps instalados e configurações do usuário serão mantidos nesta pasta do host.
- `androidboot.use_memfd=1`: **Crítico**. Instrui o Android a usar `memfd` em vez de `ashmem`. Isso evita falhas em kernels > 5.18 onde `ashmem` não existe.
- `androidboot.redroid_gpu_mode=guest`: Como a instância T4g não possui GPU dedicada, este modo força a renderização via software (CPU).
 - Nota: O Redroid usará bibliotecas como **SwiftShader** para traduzir chamadas OpenGL ES para instruções de CPU. Isso consome processamento significativo.³¹
- `androidboot.redroid_width=720`: Força a resolução para 720p (HD). Tentar renderizar 1080p (FHD) via software em apenas 2 vCPUs causará lentidão extrema e *lag* na interface. 720p é o "sweet spot" para testes funcionais em instâncias econômicas.
- `androidboot.redroid_fps=30`: Limita a taxa de quadros para economizar ciclos de CPU. 60 FPS é desnecessário para automação e testes funcionais.

Para Instâncias t3.large (x86_64):

Substitua a imagem por `redroid/redroid:11.0.0-latest`. Se precisar rodar apps ARM, você precisará de uma imagem customizada com `libhoudini` integrado, mas esteja ciente da complexidade e instabilidade mencionadas na seção 2.2.

5. Acesso Remoto, Visualização e Áudio

Como a instância EC2 é *headless* (não possui monitor físico nem interface gráfica instalada), a interação com o Android deve ser feita remotamente.

5.1. Conexão ADB (Android Debug Bridge)

No computador do desenvolvedor (local):

1. Certifique-se que o adb está instalado (parte do Android SDK Platform-Tools).
2. Conecte-se ao IP da instância:

```
Bash
adb connect <IP-PUBLICO-EC2>:5555
```

3. Verifique a conexão:

```
Bash
adb devices
# Deve listar: <IP>:5555 device
```

5.2. Visualização de Tela (Scrcpy)

O **Scrcpy** (Screen Copy) é a ferramenta padrão-ouro para esta tarefa. Ele injeta um servidor .jar no dispositivo Android que captura o vídeo, codifica em H.264 e envia via stream para o cliente, que decodifica e exibe.

Comando otimizado para WAN (Internet):

```
Bash
```

```
scrcpy --serial <IP-PUBLICO-EC2>:5555 --bit-rate 2M --max-size 800 --max-fps 30
--no-audio
```

- `--bit-rate 2M`: Limita a largura de banda a 2 Mbps. Essencial para evitar *frames* perdidos em conexões instáveis ou com latência (a AWS cobra por saída de dados, então isso também economiza custos de tráfego de rede - Data Transfer Out).
 - `--no-audio`: O encaminhamento de áudio do Android 11+ via Scrcpy exige configuração complexa ou Android 12+. Desabilitar economiza banda se o áudio não for crítico para o teste.
-

6. Desempenho, Gerenciamento de Créditos e Troubleshooting

A utilização de instâncias da família T (Burstable) exige uma gestão cuidadosa dos recursos de computação.

6.1. O Mecanismo de Créditos de CPU (T3/T4g)

As instâncias t4g.large operam com um conceito de "Linha de Base" (*Baseline*) e "Créditos de Explosão" (*Burst Credits*).

- **Baseline:** Para uma t4g.large, a AWS garante 40% de utilização de uma vCPU continuamente.¹⁸
- **Boot do Android:** O processo de inicialização do Android (boot) é intensivo em CPU, frequentemente consumindo 100% das vCPUs por 1 a 2 minutos. Isso consome créditos acumulados.
- **Renderização:** Se houver animações constantes na tela, o processo de renderização por software manterá a CPU em alta utilização, drenando créditos rapidamente.
- **Risco de Esgotamento:** Se os créditos acabarem, a AWS limitará a CPU à linha de base (40%). Isso fará o Android engasgar severamente ou travar.
- **Mitigação:** Utilize o modo **Unlimited** (ativado por padrão na maioria das T3/T4g novas). Isso permite que a instância sustente alta performance pagando uma taxa extra por vCPU-hora excedente. Monitore a métrica CPUCreditBalance no CloudWatch. Se o saldo estiver constantemente zero, considere migrar para uma instância de performance fixa como m6g.large ou c6g.large (embora sejam mais caras, oferecem previsibilidade).

6.2. Troubleshooting: Tela Preta (Black Screen)

Um dos problemas mais comuns relatados é o Scrcpy conectar, mas exibir apenas uma tela preta.³³

Diagnóstico e Correção:

1. **Causa 1: Falha no Binder/Ashmem:** O Android iniciou o kernel, mas o *userspace* (SurfaceFlinger) falhou ao inicializar por não conseguir alocar memória compartilhada.
 - **Ação:** Verifique os logs do container: `docker logs redroid-instance`. Se vir erros sobre `/dev/binder` ou `ashmem`, revise a seção 4.2 e 4.3 (especialmente a flag `use_memfd`).
2. **Causa 2: Problema na Renderização Gráfica:** O modo "guest" não está funcionando corretamente com a biblioteca gráfica.
 - **Ação:** Tente forçar o renderizador `SwiftShader` explicitamente se a imagem suportar, ou alterne o modo GPU para `auto`. Verifique se o pacote `libgl1-mesa-glx` está instalado no host, embora o container deva ser autocontido.
3. **Causa 3: Permissões de Arquivo:** O Docker não conseguiu montar `/data` corretamente.
 - **Ação:** Verifique as permissões da pasta `~/data-android` no host (`chmod 777`).

temporariamente para testar).

7. Análise Financeira Detalhada

A tabela abaixo projeta os custos mensais estimados para um ambiente em execução contínua (730 horas/mês) na região US East (N. Virginia), excluindo impostos e transferência de dados.

Componente	Opção A: T3.large (x86)	Opção B: T4g.large (ARM) - Recomendada	Opção C: Genymotion SaaS (PaaS)	Opção D: C5.metal (Bare Metal)
Computação (EC2)	\$60.74	\$49.06	\$49.06 (baseado em T4g)	~\$2,978.40
Armazenamento (30GB gp3)	\$2.40	\$2.40	\$2.40	\$2.40
Licença de Software	\$0.00 (Open Source)	\$0.00 (Open Source)	~\$365.00 (\$0.50/h) ³⁵	\$0.00
Custo Total Mensal	\$63.14	\$51.46	\$416.46	~\$2,980.80
Custo por Hora	~\$0.086	~\$0.070	~\$0.57	~\$4.08

Interpretação Econômica:

- A solução **Opção B (T4g + Redroid)** é a mais econômica, representando uma economia de **~19%** sobre a T3 e **~87%** sobre a solução SaaS da Genymotion.
 - A solução *Bare Metal* (Opção D) é financeiramente inviável para este escopo, custando quase 60 vezes mais que a solução containerizada.
-

8. Conclusão

A pesquisa confirma que a execução de um emulador Android em instâncias AWS EC2 da família t3.large ou t4g.large é **totalmente viável**, desde que se abandone a abordagem de virtualização aninhada (AVD/KVM) em favor da **containerização (Redroid)**.

A estratégia recomendada é a utilização da família **t4g.large (Graviton/ARM64)** rodando **Ubuntu 22.04 LTS**. Esta combinação alinha a arquitetura do hardware da nuvem com a arquitetura nativa do Android, eliminando a sobrecarga de tradução binária, reduzindo custos e simplificando a pilha de software. A configuração exige atenção especial aos módulos de kernel (binder/memfd), mas uma vez estabelecida, oferece um ambiente robusto, performático e escalável para testes e automação móvel a uma fração do custo das soluções tradicionais.

Referências citadas

1. Amazon EC2 instance types - Amazon Elastic Compute Cloud - AWS Documentation, acessado em janeiro 26, 2026, <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instance-types.html>
2. Nested virtualisation support on EC2 compute optimized instances | AWS re:Post, acessado em janeiro 26, 2026, <https://repost.aws/questions/QUkOwmVhagQbOumNhdfc4YcA/nested-virtualisation-support-on-ec2-compute-optimized-instances>
3. Nested Virtualization in aws bare Metal c5 instances [closed] - Stack Overflow, acessado em janeiro 26, 2026, <https://stackoverflow.com/questions/62558791/nested-virtualization-in-aws-bare-metal-c5-instances>
4. What AWS instances can support nested virtualisation?, acessado em janeiro 26, 2026, https://repost.aws/questions/QUXqOvN3_dR-Gsv5CJC6rBpQ/what-aws-instances-can-support-nested-virtualisation
5. Installing and running Android Emulator on Amazon AWS EC2 (Ubuntu 16.04 / m5.xlarge), acessado em janeiro 26, 2026, <https://gist.github.com/atyachin/cf1690085173e1fab07b9acc7af3de6>
6. USE Redroid - Radxa Docs, acessado em janeiro 26, 2026, <https://docs.radxa.com/en/rock5/rock5b/app-development/redroid>
7. ARM Native Android environment now on AWS - Genymotion, acessado em janeiro 26, 2026, <https://www.genymotion.com/blog/just-launched-arm-native-android-in-the-cloud-environment-on-aws/>
8. [Bug Report] Redroid16 failed on AWS Ubuntu 22.04 and 24.04 ..., acessado em janeiro 26, 2026, <https://github.com/remote-android/redroid-doc/issues/869>
9. redroid-doc/deploy/README.md at master - GitHub, acessado em janeiro 26, 2026, <https://github.com/remote-android/redroid-doc/blob/master/deploy/README.md>

10. How do I install the Ashmem and Binder kernel mods for Anbox?, acessado em janeiro 26, 2026,
<https://unix.stackexchange.com/questions/773524/how-do-i-install-the-ashmem-and-binder-kernel-mods-for-anbox>
11. Anbox Cloud - Ubuntu documentation, acessado em janeiro 26, 2026,
<https://documentation.ubuntu.com/anbox-cloud/explanation/anbox-cloud/>
12. How do I install ashmem and binder for anbox? : r/pop_os - Reddit, acessado em janeiro 26, 2026,
https://www.reddit.com/r/pop_os/comments/z9grg1/how_do_i_install_ashmem_and_binder_for_anbox/
13. How to install libhoudini on a custom Android x86 rig - Stack Overflow, acessado em janeiro 26, 2026,
<https://stackoverflow.com/questions/49634762/how-to-install-libhoudini-on-a-custom-android-x86-rig>
14. Install ReDroid for playing Android games in Docker containers on Linux PC, with GPU acceleration · Ivon's Blog, acessado em janeiro 26, 2026,
<https://ivonblog.com/en-us/posts/redroid-android-docker/>
15. Can't get libhoudini to work with android 13. · Issue #685 · remote-android/redroid-doc, acessado em janeiro 26, 2026,
<https://github.com/remote-android/redroid-doc/issues/685>
16. New – Amazon EC2 G5g Instances Powered by AWS Graviton2 Processors and NVIDIA T4G Tensor Core GPUs, acessado em janeiro 26, 2026,
<https://aws.amazon.com/blogs/aws/new-amazon-ec2-g5g-instances-powered-by-aws-graviton2-processors-and-nvidia-t4g-tensor-core-gpus/>
17. AWS Launches First NVIDIA GPU-Accelerated Graviton-Based Instance with Amazon EC2 G5g, acessado em janeiro 26, 2026,
<https://developer.nvidia.com/blog/aws-launches-first-nvidia-gpu-accelerated-graviton-based-instance-with-amazon-ec2-g5g/>
18. Amazon EC2 T4g Instances, acessado em janeiro 26, 2026,
<https://aws.amazon.com/ec2/instance-types/t4/>
19. AWS T3 Vs. T3a Vs. T4g Instances: A 2024 Comparison - CloudZero, acessado em janeiro 26, 2026, <https://www.cloudzero.com/advisor/t3-vs-t3a-vs-t4g/>
20. cost savings : r/aws - Reddit, acessado em janeiro 26, 2026,
https://www.reddit.com/r/aws/comments/1gjc84g/cost_savings/
21. AWS Marketplace: Anbox Cloud Appliance - Arm - Amazon.com, acessado em janeiro 26, 2026,
<https://aws.amazon.com/marketplace/pp/prodview-aqmdt52vqs5qk>
22. AWS Marketplace: Anbox_cloud, acessado em janeiro 26, 2026,
<https://aws.amazon.com/marketplace/pp/prodview-r5svkxfq63p6u>
23. linux - how do I install ashmem or binder? - Super User, acessado em janeiro 26, 2026,
<https://superuser.com/questions/1209153/how-do-i-install-ashmem-or-binder>
24. Cannot install anbox kernel modules #2076 - GitHub, acessado em janeiro 26, 2026, <https://github.com/anbox/anbox/issues/2076>
25. [Feature Request] Support memfd · Issue #841 · remote-android/redroid-doc -

- GitHub, acessado em janeiro 26, 2026,
<https://github.com/remote-android/redroid-doc/issues/841>
26. ASHMEM removed from Linux 5.18 · Issue #189 · remote-android/redroid-doc - GitHub, acessado em janeiro 26, 2026,
<https://github.com/remote-android/redroid-doc/issues/189>
 27. ASHMEM removed from Linux 5.18 · Issue #406 - GitHub, acessado em janeiro 26, 2026, <https://github.com/waydroid/waydroid/issues/406>
 28. remote-android/redroid-modules: redroid kernel modules (binderfs, ashmem etc.) - GitHub, acessado em janeiro 26, 2026,
<https://github.com/remote-android/redroid-modules>
 29. linux-modules-extra-6.8.0-1040-aws : Jammy (22.04) : Ubuntu - Launchpad, acessado em janeiro 26, 2026,
<https://launchpad.net/ubuntu/jammy/+package/linux-modules-extra-6.8.0-1040-aws>
 30. Streaming Android games from cloud to mobile with AWS Graviton-based Amazon EC2 G5g instances, acessado em janeiro 26, 2026,
<https://aws.amazon.com/blogs/compute/streaming-android-games-from-cloud-to-mobile-with-aws-graviton-based-amazon-ec2-g5g-instances/>
 31. Running headless android emulator on AWS EC2 Ubuntu instance (ARM64 / aarch64) - 2022 - GitHub Gist, acessado em janeiro 26, 2026,
<https://gist.github.com/atyachin/2f7c6054c4cd6945397165a23623987d>
 32. Setting-up Android Emulator + Appium Driver + noVNC Client on Headless Debian 12 Linux VM | by Amit Srivastava | Dec, 2025 | Medium, acessado em janeiro 26, 2026,
<https://medium.com/@amitsriv99/setting-up-android-emulator-appium-driver-no-vnc-on-headless-debian-12-compute-engine-e5811c997310>
 33. Troubleshoot issues connecting to your Amazon EC2 Windows instance, acessado em janeiro 26, 2026,
<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/troubleshoot-connect-windows-instance.html>
 34. redroid11 scrcpy black screen · Issue #427 · remote-android/redroid-doc - GitHub, acessado em janeiro 26, 2026,
<https://github.com/remote-android/redroid-doc/issues/427>
 35. AWS Marketplace: Genymotion Cloud - Android 12.0 (arm64) - Amazon.com, acessado em janeiro 26, 2026,
<https://aws.amazon.com/marketplace/pp/prodview-2xf5cs3mbvnso>